

HERRAMIENTA CASE COLABORATIVA CON INTELIGENCIA ARTIFICIAL

Diseño Conceptual de Datos UML 2.5+, Generación de Código Spring Boot 3.4 +
PostgreSQL (OMT/DMO) e Interoperabilidad OMG XMI 2.1

Sistema CASE Studio AI con Asistencia de Voz, Visión y App Móvil Flutter Offline con SLM On-Device

Materia:	Sistemas de Información II / Ingeniería de Software I
Docente:	Ing. David Morales
Estudiante / Autor:	César Quispe Delgado
Usuario GitHub:	julioficctr7 (https://github.com/julioficctr7)
Cuenta AWS:	Nombre: cesar ID: 8865-9557-5652
Repositorio Oficial:	https://github.com/julioficctr7/case-studio-ai
Fecha de Entrega:	23 de Septiembre de 2026 — Santa Cruz de la Sierra, Bolivia
Ejemplar:	COPIA 1 — REVISIÓN OFICIAL 8:00 AM (DOCENTE)

HERRAMIENTA CASE COLABORATIVA CON INTELIGENCIA ARTIFICIAL

Diseño Conceptual de Datos UML 2.5+, Generación de Código Spring Boot 3.4 +
PostgreSQL (OMT/DMO) e Interoperabilidad OMG XMI 2.1

Sistema CASE Studio AI con Asistencia de Voz, Visión y App Móvil Flutter Offline con SLM On-Device

Materia:	Sistemas de Información II / Ingeniería de Software I
Docente:	Ing. David Morales
Estudiante / Autor:	César Quispe Delgado
Usuario GitHub:	julioficctcr7 (https://github.com/julioficctcr7)
Cuenta AWS:	Nombre: cesar ID: 8865-9557-5652
Repositorio Oficial:	https://github.com/julioficctcr7/case-studio-ai
Fecha de Entrega:	23 de Septiembre de 2026 — Santa Cruz de la Sierra, Bolivia
Ejemplar:	COPIA 2 — ARCHIVO ACADÉMICO 15:45 PM (BIBLIOTECA)

ÍNDICE GENERAL Y ESTRUCTURA METODOLÓGICA

PARTE I — PERFIL DEL PROYECTO	Pág. 3
1.1 Introducción y Planteamiento del Problema	Pág. 3
1.2 Objetivo General y Objetivos Específicos	Pág. 3
1.3 Alcance de los 5 Subsistemas del Monorepo	Pág. 4
PARTE II — FUNDAMENTO TEÓRICO RIGUROSO	Pág. 5
2.1 Ingeniería CASE y CBSE (Clemens Szyperski, 1998)	Pág. 5
2.2 Arquitectura de Software y la Ley de Longevidad ('El Dinosaurio')	Pág. 6
2.3 Taxonomía OMG UML 2.5+ y el Linaje Booch + Rumbaugh (OMT) + Jacobson (OOSE)	Pág. 6
2.4 Mapeo Relacional OMT/DMO y Reglas de Edgar F. Codd (1NF a 3NF)	Pág. 7
2.5 Spring Boot 3.4.x, Hibernate ORM y Arquitectura en 5 Capas Limpias	Pág. 8
2.6 Cliente Móvil Flutter con SLM Local On-Device (lib_llama_cpp / Qwen GGUF)	Pág. 8
2.7 Agente Tutor Interactivo Co-Pilot CASE (Sensor de Heurísticas)	Pág. 9
PARTE III — METODOLOGÍA PROCESO UNIFICADO (PUDS)	Pág. 10
3.1 Flujo de Requisitos: Matriz de los 15 Casos de Uso Canónicos	Pág. 10
3.2 Flujo de Análisis: Robustez BCE y Diagramas de Comunicación	Pág. 12
3.3 Flujo de Diseño: Secuencia, Despliegue AWS EC2 y Tabla de Volumetría	Pág. 13
3.4 Flujo de Implementación: Stack Real Angular 22 + NestJS + Mustache	Pág. 15
3.5 Flujo de Pruebas: Matriz de Aceptación de los 15 Casos de Uso	Pág. 16
PARTE IV — ENLACES DE ACCESO, CÓDIGOS QR Y DEFENSA	Pág. 17
4.1 Repositorio GitHub con Pipeline CI/CD	Pág. 17
4.2 Servidor Cloud en Producción AWS EC2	Pág. 17
4.3 Guía de Defensa en Vivo: Las 3 Entradas, Fusión de AST y Exclusión Mutua	Pág. 18

PARTE I — PERFIL DEL PROYECTO

1.1 Introducción y Descripción del Problema: En la industria del software moderna, la fase de diseño conceptual de datos constituye el cimiento estructural de cualquier sistema de gestión empresarial. Errores u omisiones en el diagrama de clases conllevan esquemas relacionales anómalos (inconsistencias por redundancia, anomalías de inserción, borrado y actualización). Las herramientas CASE tradicionales sufren de rigidez de interfaz, carecen de concurrencia colaborativa en tiempo real y no integran motores de inteligencia artificial multimodal capaces de traducir voz o bocetos ópticos a modelos formales sin romper el flujo de trabajo.

1.2 Objetivo General: Desarrollar una plataforma CASE colaborativa integral orientada al modelado conceptual de diagramas de clases UML 2.5+, con soporte de entrada multimodal (GUI tradicional, comandos de voz y visión artificial por webcam), validación formal relacional de 1NF a 3NF, interoperabilidad OMG XMI 2.1 con Enterprise Architect, generación automatizada de backends en Spring Boot 3.4.x con PostgreSQL y producción de un cliente de pruebas móvil en Flutter con IA local on-device 100% offline.

1.3 Objetivos Específicos:

- Implementar un lienzo grafocéntrico reactivo en Angular 22 (@foblex/flow) con exclusión mutua de nodos en tiempo real (nodeLocks) vía WebSockets STOMP.
- Construir un motor de compilación dirigido por modelos (MDD) que transforme el Árbol de Sintaxis Abstracta (AST) a microservicios Spring Boot en 5 capas desacopladas con Maven Wrapper (.mvnw).
- Proveer interoperabilidad bidireccional XMI 2.1/2.5 para intercambio de esquemas con Enterprise Architect y StarUML.
- Incorporar un agente tutor interactivo (Co-Pilot CASE) visible en UI que asista didácticamente al usuario en la resolución de anomalías de normalización.
- Desplegar el sistema en infraestructura cloud AWS EC2 mediante Docker Compose, Nginx Reverse Proxy y certificados SSL automáticos Let's Encrypt.

PARTE II — FUNDAMENTO TEÓRICO RIGUROSO

2.1 Ingeniería CASE y Desarrollo Basado en Componentes (CBSE): Siguiendo a Clemens Szyperski (1998, *Component Software: Beyond Object-Oriented Programming*), un componente de software se define formalmente como 'una unidad de composición con contratos de interfaz explícitos y dependencias de contexto bien determinadas, capaz de ser desplegada de forma independiente y sujeta a composición por terceros'. La plataforma CASE Studio AI se clasifica como un entorno **Integrated-CASE (I-CASE)** al integrar armónicamente las fases Upper-CASE (modelado conceptual y análisis de requisitos) con las fases Lower-CASE (generación automática de código ejecutable, scripts DDL SQL y arneses de prueba Postman).

2.2 Arquitectura de Software y la Ley de Longevidad ('El Dinosaurio'): El propósito supremo de la arquitectura de software es garantizar la longevidad del sistema, permitiendo que el software sobreviva a las modas pasajeras de frameworks y hardware. Una arquitectura deficiente incurre en el síndrome del 'dinosaurio en el pantano': cualquier modificación menor en la lógica de negocio desestabiliza módulos periféricos, encareciendo el mantenimiento. La adopción del patrón en 5 Capas Desacopladas (Controller, Service, Repository, Entity, DTO) garantiza que cada componente absorba los cambios sin propagar efectos colaterales.

2.3 Taxonomía OMG UML 2.5+ y el Linaje Metodológico: El estándar UML unifica las metodologías de Grady Booch (Diseño Orientado a Objetos), James Rumbaugh (Técnica de Modelado de Objetos - OMT) e Ivar Jacobson (Ingeniería de Software Orientada a Objetos - OOSE). Nuestro modelador implementa la especificación canónica OMG UML 2.5+: clases con compartimentos para atributos tipados y operaciones, visibilidad (+, -, #, ~),

multiplicidades formales y relaciones con semántica estricta (Asociación, Agregación con rombo hueco, Composición con rombo relleno, Generalización y Dependencia).

2.4 Mapeo Relacional OMT/DMO y Reglas de Edgar F. Codd (1NF a 3NF):

- **Primera Forma Normal (1NF):** Todo atributo es atómico; cada entidad posee una Clave Primaria unívoca ({PK} / @Id) sin grupos repetitivos.
- **Segunda Forma Normal (2NF):** Cumple 1NF y ningún atributo no clave depende parcialmente de una clave primaria compuesta. En relaciones 1..*, la Clave Foránea ({FK}) se sitúa estrictamente en la entidad del extremo muchos (*).
- **Tercera Forma Normal (3NF):** Cumple 2NF y ningún atributo no clave depende transitivamente de la clave primaria. Toda relación *.* se descompone en dos relaciones 1..* con tabla asociativa intermedia.

2.5 Backend Spring Boot 3.4.x y Java 21 LTS: El código generado sigue estrictamente la arquitectura en capas con Jakarta Persistence y Spring Data JPA sobre PostgreSQL. Se descarta tajantemente la versión alucinada 'Spring Boot 4.1.0' (inexistente), garantizando compilación inmediata con Maven Wrapper (./mvnw spring-boot:run).

2.6 Cliente Móvil Flutter con SLM Local On-Device: Generación de app móvil en Flutter con arquitectura BLoC, SQLite local y soporte offline mediante modelos de lenguaje reducidos (SLM) cuantizados a 4 bits (Qwen / Gemma vía lib_llama_cpp), operando sin conexión a internet.

2.7 Agente Tutor Interactivo Co-Pilot CASE: Componente visible en la esquina inferior del lienzo que actúa como asistente pedagógico, detectando omisiones heurísticas en tiempo real (clases sin PK, relaciones huérfanas) y guiando al estudiante en el uso óptimo de la herramienta.

PARTE III — METODOLOGÍA PROCESO UNIFICADO (PUDS)

3.1 Flujo de Captura de Requisitos — Matriz de los 15 Casos de Uso Canónicos:

CU	Nombre del Caso de Uso	Actor	Ciclo PUDS
CU01	Autenticación Stateless y Gestión de Cuentas	Usuario	Ciclo 1: Core
CU02	Control de Roles (RBAC) y Gobernanza	Super Admin	Ciclo 1: Core
CU03	Gestión de Espacios de Trabajo y Plantillas	Arquitecto	Ciclo 1: Core
CU04	Auditoría Forense y Trazabilidad (diagram_history)	Auditor	Ciclo 1: Core
CU05	Modelado Canónico de Clases y Relaciones UML 2.5+	Arquitecto	Ciclo 1: Core
CU06	Certificación Heurística de Normalización (1NF–3NF)	Arquitecto	Ciclo 1: Core
CU07	Agente Tutor Interactivo y Onboarding (Co-Pilot CASE)	Aprendiz	Ciclo 1: Core
CU08	Exportación Multiformato (XMI/PNG/PDF/Excel)	Arquitecto	Ciclo 2: MDD
CU09	Importación XMI 2.1 (ArchiTec / Enterprise Architect)	Arquitecto	Ciclo 2: MDD
CU10	Generación Backend Spring Boot 3.4 (5 Capas + DDL)	Arquitecto	Ciclo 2: MDD
CU11	Generación Suite Postman v2.1 y Swagger OpenAPI	Tester	Ciclo 2: MDD
CU12	Generación Cliente Móvil Flutter con SLM On-Device	Dev Móvil	Ciclo 2: MDD
CU13	Modelado por Dictado de Voz (Mutación Atómica AST)	Arquitecto	Ciclo 3: IA
CU14	Digitalización Óptica por Visión (Detección Duplicados)	Arquitecto	Ciclo 3: IA
CU15	Sincronización Concurrente con Exclusión Mutua (nodeLocks)	Colaborador	Ciclo 3: IA

3.2 Diseño Físico de Base de Datos y Tabla de Volumetría Proyectada:

Tabla Física	Bytes/Reg	Reg. Inicial	Crecim. Anual	Volumen (MB/año)
user_profiles	512 B	100	+500	0.25 MB
diagram_projects	1,024 B	50	+1,000	1.02 MB
class_nodes	2,048 B	500	+15,000	30.72 MB
relationships	512 B	750	+20,000	10.24 MB
diagram_history	8,192 B	1,000	+50,000	409.60 MB
audit_logs	1,024 B	2,000	+100,000	102.40 MB
domain_templates	4,096 B	20	+10	0.04 MB
ai_prompt_logs	2,048 B	500	+30,000	61.44 MB
collaboration_messages	512 B	2,000	+100,000	51.20 MB
normalization_certifications	4,096 B	50	+1,000	4.10 MB
TOTAL CONSOLIDADO	—	—	—	~672.85 MB / año

PARTE IV — ENLACES OFICIALES, CÓDIGOS QR Y AUDITORÍA DE DEFENSA

1. REPOSITORIO GITHUB CI/CD

<https://github.com/julioficctr7/case-studio-ai>

Código fuente completo del monorepo (Angular 22, NestJS, Spring Boot 3.4 generator, Flutter templates, Dockerfiles y workflows de GitHub Actions).



2. APLICACIÓN EN PRODUCCIÓN CLOUD (AWS)

<https://casestudio.duckdns.org>

Instancia AWS EC2 en cuenta **cesar (8865-9557-5652)** protegida con Nginx Reverse Proxy y certificado SSL HTTPS Let's Encrypt verificado.



Rúbrica de Defensa en Vivo (Paso 4):

1. **Las 3 Entradas:** Comandos de voz con Web Speech API aplican mutaciones atómicas al AST sin re-renderizar todo el lienzo (evita degradación de performance). La visión por webcam procesa bocetos manuscritos extrayendo clases y atributos en tiempo real.
2. **Blindaje contra Duplicados:** Si el usuario o la cámara intentan agregar una clase preexistente (ej. 'Cliente'), el motor de AST Merging fusiona los nuevos atributos o emite advertencia de colisión; **jamás crea dos nodos duplicados**.
3. **Backend Spring Boot Compilable:** El ZIP descargable incluye Maven Wrapper (./mvnw) y compila inmediatamente sin errores. Los métodos del diagrama UML no se confunden con columnas de PostgreSQL.
4. **Cliente Móvil Flutter:** Funciona 100% offline con base de datos SQLite local y motor SLM on-device para registrar operaciones sin señal.